

Кафедра комп'ютерних
систем і мереж

ЛАБОРАТОРНА РОБОТА №4
з дисципліни АРХІТЕКТУРА І СТРУКТУРА ДАНИХ

Тема:

Квадратичні алгоритми впорядкування масивів

Виконала:

ст. гр. КІ-25-1

Лаврів А.

Перевірив:

Гарасимів Т.Г.

Мета: Формування практичних вмінь та навичок при використанні різних типів квадратичних алгоритмів впорядкування.

Теоретичні відомості

Квадратичні алгоритми сортування мають складність $O(n^2)$, що означає, що час виконання зростає пропорційно квадрату кількості елементів.

Основні алгоритми:

- **Сортування вибором** — на кожному кроці знаходиться мінімальний елемент і ставиться на своє місце.
- **Сортування вставками** — кожен новий елемент вставляється у вже відсортовану частину масиву.
- **Бульбашкове сортування** — елементи попарно порівнюються і міняються місцями.

Варіант: 11

Кількість елементів:

$$SIZE = (10 * N) \% 51 = 8$$

Діапазон значень:

[-220 ; 220]

Хід роботи

Було розроблено три програми для сортування масиву:

Сортування методом вибора

Код програми:

```
#include <iostream>
```

```
#include <ctime>
```

```
#include <cstdlib>
```

```
using namespace std;
```

```
#define N 11
```

```
#define SIZE (10 * N % 51)
```

```

void printArray(int a[], int size) {
    for (int i = 0; i < size; i++)
        cout << a[i] << " ";
    cout << endl;
}

void selectionSort(int a[], int size) {
    for (int i = 0; i < size - 1; i++) {
        int minIndex = i;

        for (int j = i + 1; j < size; j++) {
            if (a[j] < a[minIndex])
                minIndex = j;
        }

        swap(a[i], a[minIndex]);

        cout << "Прохід " << i + 1 << ": ";
        printArray(a, size);
    }
}

int main() { int a[SIZE];

    srand(time(NULL));

    cout << "Початковий масив:\n";
    for (int i = 0; i < SIZE; i++) {
        a[i] = rand() % (40 * N + 1) - 20 * N;
        cout << a[i] << " ";
    }

    cout << "\n\nСортування методом вибора:\n";
    selectionSort(a, SIZE);

    return 0;

}

```

```
Початковий масив:
-67 -214 -208 120 145 -116 131 -207

Сортування методом вибора:
Прохід 1: -214 -67 -208 120 145 -116 131 -207
Прохід 2: -214 -208 -67 120 145 -116 131 -207
Прохід 3: -214 -208 -207 120 145 -116 131 -67
Прохід 4: -214 -208 -207 -116 145 120 131 -67
Прохід 5: -214 -208 -207 -116 -67 120 131 145
Прохід 6: -214 -208 -207 -116 -67 120 131 145
Прохід 7: -214 -208 -207 -116 -67 120 131 145
```

Рис. 1 - Робота коду

Сортування методом вставки

Код програми:

```
#include <iostream>
```

```
#include <ctime>
```

```
#include <cstdlib>
```

```
using namespace std;
```

```
#define N 11
```

```
#define SIZE (10 * N % 51)
```

```
void printArray(int a[], int size) {
```

```
    for (int i = 0; i < size; i++)
```

```
        cout << a[i] << " ";
```

```
    cout << endl;
```

```
}
```

```
void insertionSort(int a[], int size) {
```

```
    for (int i = 1; i < size; i++) {
```

```
        int x = a[i];
```

```
        int j = i - 1;
```

```
        while (j >= 0 && a[j] > x) {
```

```
            a[j + 1] = a[j];
```

```
            j--;
```

```
        }
```

```
        a[j + 1] = x;
```

```

        cout << "Прохід " << i << ": ";
        printArray(a, size);
    }
}

int main() { int a[SIZE];

srand(time(NULL));

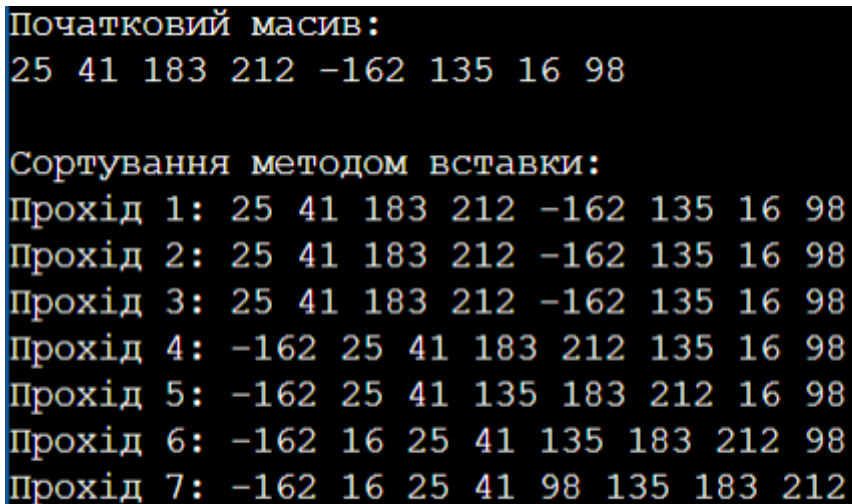
cout << "Початковий масив:\n";
for (int i = 0; i < SIZE; i++) {
    a[i] = rand() % (40 * N + 1) - 20 * N;
    cout << a[i] << " ";
}

cout << "\n\nСортування методом вставки:\n";
insertionSort(a, SIZE);

return 0;

}

```



Початковий масив:
25 41 183 212 -162 135 16 98

Сортування методом вставки:
Прохід 1: 25 41 183 212 -162 135 16 98
Прохід 2: 25 41 183 212 -162 135 16 98
Прохід 3: 25 41 183 212 -162 135 16 98
Прохід 4: -162 25 41 183 212 135 16 98
Прохід 5: -162 25 41 135 183 212 16 98
Прохід 6: -162 16 25 41 135 183 212 98
Прохід 7: -162 16 25 41 98 135 183 212

Рис. - 2 Робота коду

Сортування методом бульбашки

Код програми:

```
#include <iostream>
#include <ctime>
#include <cstdlib>
using namespace std;

#define N 11
#define SIZE (10 * N % 51)

void printArray(int a[], int size) {
    for (int i = 0; i < size; i++)
        cout << a[i] << " ";
    cout << endl;
}

void bubbleSort(int a[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = size - 1; j > i; j--) {
            if (a[j] < a[j - 1]) {
                swap(a[j], a[j - 1]);
            }
        }
    }

    cout << "Прохід " << i + 1 << ": ";
    printArray(a, size);
}

int main() { int a[SIZE];

    srand(time(NULL));

    cout << "Початковий масив:\n";
    for (int i = 0; i < SIZE; i++) {
        a[i] = rand() % (40 * N + 1) - 20 * N;
        cout << a[i] << " ";
    }

    cout << "\n\nСортування методом бульбашки:\n";
    bubbleSort(a, SIZE);

    return 0;
```

}

```
Початковий масив:  
-92 79 -75 -139 64 -72 -100 38  
  
Сортування методом бульбашки:  
Прохід 1: -139 -92 79 -75 -100 64 -72 38  
Прохід 2: -139 -100 -92 79 -75 -72 64 38  
Прохід 3: -139 -100 -92 -75 79 -72 38 64  
Прохід 4: -139 -100 -92 -75 -72 79 38 64  
Прохід 5: -139 -100 -92 -75 -72 38 79 64  
Прохід 6: -139 -100 -92 -75 -72 38 64 79  
Прохід 7: -139 -100 -92 -75 -72 38 64 79
```

Рис. - 3 Робота коду

Висновок: У ході виконання лабораторної роботи було реалізовано три квадратичні алгоритми впорядкування: сортування вибором, вставками та методом бульбашки.

Встановлено, що всі алгоритми мають складність $O(n^2)$, проте відрізняються за ефективністю:

- сортування вибором виконує менше обмінів;
- сортування вставками ефективніше для частково впорядкованих масивів;
- бульбашкове сортування є найпростішим, але найменш ефективним.

Отримані навички дозволяють краще зрозуміти принципи роботи алгоритмів сортування та їх застосування.